NS3SIONNA: MIMO-ABSTRAKTIONSMODELL FÜR IEEE 802.11AX/BE

Stand: 19. Juli 2026

Arbeitszweig: feature/su-mimo-minimal

Basis: ns-3.40, ns3sionna c896b858; Auftragsbasis 46d73fa; Sionna 1.2.1

1. ZIEL UND ABGRENZUNG

Sionna RT berechnet die komplexe, frequenzselektive
Kanalübertragungsmatrix. Die eigentliche Link-Abstraktion, also
Post-Processing-SINR, Effective-SINR, MCS/PER und später die Auswahl von
Precoding und MU-MIMO, wird ausschließlich in ns-3 ausgeführt.

Die Entwicklung erfolgt in kleinen, einzeln testbaren Schritten. Der aktuelle
Stand implementiert die Unterphasen 1.1 bis 1.4: numerischen SU-MIMO-Kern,
konfigurierbare Sionna-Arrays, einen dimensionssicheren MIMO-Datenvertrag,
explizit kalibrierbare 802.11ax-EESM-/PER-Abbildung und die schmale Einkopplung
in den SpectrumWifiPhy-Empfangspfad. Ein reproduzierbares 1x1-/2x2-Beispiel
deckt Vollrang- und Rang-1-Kanäle ab. Phase 2 und jede Form von Precoding oder
Beamforming wurden nicht begonnen.

2. REPRODUZIERBARE AUSGANGSBASIS

Die lokale Referenzplattform ist CPU-only:

  - Ubuntu 24.04, GCC 13.3, CMake 3.28
  - ns-3.40 mit Tests und Beispielen
  - ZeroMQ 4.3.5 und Protocol Buffers 3.21.12
  - Python 3.12.12, Sionna 1.2.1, NumPy 1.26.4
  - TensorFlow erkennt keine GPU und verwendet den CPU-Pfad

Ohne explizite Arraykonfiguration verwendet der Server weiterhin exakt die
bisherige 1x1-Sende- und Empfangsantenne. Die normalisierte SISO-CFR erfüllt

$$
  \frac{1}{N_\mathrm{sc}}\sum_k |h[k]|^2 = 1,
$$

während der Wideband-Pfadverlust separat in dB übertragen wird. Unterphase 1.2
verallgemeinert diese Trennung auf alle Antennenpaare: H[k] enthält
Small-Scale-Fading und der skalare Wideband-Verlust bestimmt die Referenz-SNR.

3. ENTWICKLUNGSPLAN

PHASE 0: BASELINE UND MESSVERTRAG

1. Versionen und CPU-Ausführung reproduzierbar festhalten.
2. Bestehende SISO-Import-, Unit- und Beispieltests ausführen.
3. Einheiten, Antennenreihenfolge, CFR-Normalisierung und
   Subcarrier-Reihenfolge als Datenvertrag festlegen.

Abnahmekriterium: SISO-Verhalten bleibt unverändert und alle alten Tests laufen.

PHASE 1: BELASTBARES SU-MIMO-MINIMALMODELL

UNTERPHASE 1.1: NUMERISCHER KERN (IMPLEMENTIERT)

- CFR-Tensor H[k] mit expliziten Dimensionen N_sc x N_rx x N_tx.
- Open-Loop Spatial Multiplexing, ein Stream je Sendeantenne.

- Gleiche Gesamtleistungsaufteilung über N_s = N_tx Streams.
- Perfekte CFR-Kenntnis am Empfänger und linearer MMSE-Detektor.
- Post-MMSE-SINR pro Subcarrier und Stream.
- Mutual-Information-basierte Kompression über Subcarrier.
- Analytische Unit-Tests für SISO, orthogonale 2x2-Kanäle und frequenzselektive Kanäle.

Abnahmekriterium: analytische Werte werden mit einer Toleranz von 1e-10 reproduziert.

UNTERPHASE 1.2: MIMO-DATENVERTRAG SIONNA -> NS-3 (IMPLEMENTIERT)

- Antennendimensionen rückwärtskompatibel in message.proto ergänzen.
- Flache Reihenfolge sc/rx/tx verbindlich festlegen.
- Sionna-Arrays konfigurierbar machen und die CFR ohne Verlust der Antennendimensionen serialisieren.
- CacheEntry auf MimoChannelMatrix umstellen.
- Python-Shape-Tests und C++-Deserialisierungstests hinzufügen.

Abnahmekriterium: 1x1 ist binär/funktional kompatibel; synthetische 2x2-Werte kommen indexgenau in ns-3 an.

UNTERPHASE 1.3: 802.11AX-LINK-ABSTRAKTION (IMPLEMENTIERT)

- Referenz-SNR aus Sendeleistung, Wideband-Verlust, Bandbreite, Rauschzahl und thermischem Rauschen berechnen.
- Effektive SINR mit einer explizit kalibrierbaren EESM-Abbildung je MCS bestimmen; ohne belegte Kalibrierung existieren bewusst keine Default-Betas.
- Vorhandene ns-3 ErrorRateModel-Schnittstellen soweit möglich wiederverwenden.
- SISO-Parität und AWGN-Kurven gegen ns-3 validieren.

Abnahmekriterium: PER-vs-SNR-Kurven sind monoton, reproduzierbar und im SISO-Grenzfall deckungsgleich zur festgelegten Referenz. Alle Kriterien wurden mit den in Abschnitt 6 beschriebenen automatischen Tests erfüllt.

UNTERPHASE 1.4: WIFIPHY-INTEGRATION UND BEISPIEL (IMPLEMENTIERT)

- Nur den schmalsten notwendigen Empfangspfad in SpectrumWifiPhy anbinden.
- NSS aus WifiTxVector verwenden und ungültige Antennenkonfigurationen früh ablehnen.
- Beispiel ns3_wifi_sumimo_minimal.cc mit 1x1/2x2-Vergleich anlegen.
- Durchsatz, PER, Stream-SINR und Kanalrang ausgeben.

Abnahmekriterium: 2x2 gewinnt auf gut konditionierten Kanälen und kollabiert auf Rang-1-Kanälen nachvollziehbar gegen 1x1.
Die automatisierten Ergebnisse in Abschnitt 6 erfüllen dieses Kriterium.

PHASE 2: HIGH-LEVEL-API

Geplant ist ein SionnaMimoHelper, der mindestens folgende Aspekte gemeinsam konfiguriert:

- Arraygeometrie: Zeilen, Spalten, Elementabstand und Polarisation
- Ausrichtung: Azimut, Elevation und Rollwinkel
- Betriebsart: SISO, SU-MIMO Spatial Multiplexing, SU-Beamforming, MU-MIMO
- Zahl der TX-/RX-Antennen, maximale NSS und Empfängertyp
- Kanalnormalisierung, Rauschzahl und Effective-SINR-Modell

Die API validiert Wi-Fi-, Sionna- und Abstraktionsparameter zentral, sodass Beispiele keine internen Objekte manuell verdrahten müssen.

PHASE 3: SU-MIMO MIT PRECODING UND BEAMFORMING

Unterphasen:

1. festes, benutzergegebenes Precoding W[k]
2. MRT/MRC für einen Stream
3. SVD-Precoding und optionale Wasserfüllung
4. quantisierte 802.11ax/be-Codebooks und veraltete CSI

Jede Unterphase erhält analytische Referenzkanäle sowie ein Beispiel
ns3_wifi_sumimo_precoding.cc.

PHASE 4: MU-MIMO

Unterphasen:

1. mehrere CFR-Matrizen H_u[k] in einem gemeinsamen Zeitstempel
2. Downlink-ZF und regularisiertes ZF
3. Inter-User-Interferenz in der Stream-SINR
4. RU/NSS-Zuordnung für HE/EHT und Scheduler-Schnittstelle
5. Uplink-MU-MIMO

Das Phasenbeispiel ns3_wifi_mumimo.cc vergleicht SU-Round-Robin mit MU-ZF bei
unterschiedlicher räumlicher Nutzerkorrelation.

PHASE 5: KALIBRIERUNG, SKALIERUNG UND REPRODUZIERBARKEIT

- PER-Kurven gegen Link-Level-Referenzen kalibrieren.
- Sensitivität gegenüber Antennenabstand, Orientierung, CSI-Alter und
Raytracing-Samples untersuchen.
- Cache- und IPC-Speicherbedarf für große N_sc, N_tx und N_rx messen.
- Deterministische Seeds, Ergebnisformate und CI-Matrix für CPU/GPU festlegen.

4. DATENMODELL UND IPC-VERTRAG DER UNTERPHASEN 1.1/1.2

4.1 TENSOR UND FLACHE REIHENFOLGE

Für jeden Subcarrier k ist

$$
\mathbf{H}[k] \in \mathbb{C}^{N_\mathrm{rx}\times N_\mathrm{tx}}.
$$

Der C++-Container und die beiden Protocol-Buffer-Felder csi_real und csi_imag
speichern die Werte flach. Der Index für den Eintrag h[k,r,t] lautet

$$
i(k,r,t) = (k N_\mathrm{rx} + r)N_\mathrm{tx} + t.
$$

Damit ist k die langsamste und t die schnellste Dimension. Es gelten die
verbindlichen Längen

$$
|\mathrm{frequencies}| = N_\mathrm{sc},
\qquad
|\mathrm{csi_real}| = |\mathrm{csi_imag}|
= N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}.
$$

Die neuen skalaren Felder num_rx_antennas und num_tx_antennas wurden mit den

Feldnummern 9 und 10 an RxNodeInfo angehängt. Der Proto3-Standardwert null wird beim Empfänger als eins interpretiert. Für eine neue 1x1-Antwort lässt der Python-Server beide Felder bewusst ungesetzt; ihre serialisierte Darstellung ist dadurch bytegleich zur alten Nachricht. Alte Sender werden als 1x1 gelesen, alte Empfänger ignorieren die unbekannten neuen Felder einer MIMO-Antwort.

4.2 TRANSFORMATION AUS SIONNA

Sionna 1.2.1 liefert die CFR in der Form

$$
H_\mathrm{raw}
[\ell,r,q,t,n,k]
$$

mit Linkindex ℓ, RX-Antenne r, TX-Index q, TX-Antenne t, Zeitindex n und Subcarrier k. Eine ns3sionna-Anfrage enthält genau einen Sender und einen Zeitpunkt. Deshalb wird ohne squeeze-Operation explizit ausgewählt:

$$
H_\ell[r,t,k] =
H_\mathrm{raw}[\ell,r,0,t,0,k].
$$

Vor der Serialisierung wird nach [k,r,t] transponiert und anschließend in C-Reihenfolge abgeflacht. Dadurch bleiben auch Singleton-Dimensionen sichtbar und 2x2-Werte landen indexgenau im C++-Container.

4.3 WIDEBAND-VERLUST UND MIMO-NORMALISIERUNG

Für die rohe Link-CFR wird die mittlere Leistung gemeinsam über Subcarrier und Antennenpaare gebildet:

$$
\bar P =
\frac{1}{N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}}
\sum_{k=0}^{N_\mathrm{sc}-1}
\sum_{r=0}^{N_\mathrm{rx}-1}
\sum_{t=0}^{N_\mathrm{tx}-1}
|H_\ell[r,t,k]|^2.
$$

Der übertragene skalare Wideband-Verlust und die normalisierte Matrix lauten

$$
L_\mathrm{wb} = -10\log_{10}(\bar P)\ \mathrm{dB},
\qquad
H_\mathrm{norm}[r,t,k] =
\frac{H_\ell[r,t,k]}{\sqrt{\bar P}}.
$$

Damit gilt

$$
\frac{1}{N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}}
\sum_{k,r,t}|H_\mathrm{norm}[r,t,k]|^2 = 1.
$$

Für N_rx=N_tx=1 ist dies exakt die bisherige SISO-Normalisierung. Die Aufteilung der gesamten Sendeleistung auf N_s Streams bleibt davon getrennt und erfolgt weiterhin im numerischen Modell mit dem Faktor 1/N_s.

4.4 ARRAYKONFIGURATION UND NS-3-CACHE

SimInitMessage besitzt optionale tx_array- und rx_array-Nachrichten mit Zeilen, Spalten, vertikalem und horizontalem Abstand in Wellenlängen, Elementmuster und Polarisation. Ohne Nachricht werden exakt die alten Arrays verwendet: 1x1/tr38901/V am Sender und 1x1/dipole/V am Empfänger. SionnaHelper bietet dafür die kleinen Methoden ConfigureTransmitArray und ConfigureReceiveArray; die umfassende knotenbezogene High-Level-API bleibt Phase 2 vorbehalten.

Die Antwortdimensionen werden aus der tatsächlich von Sionna gelieferten Form abgeleitet und berücksichtigen damit auch die Polarisation. CacheEntry hält die CFR nun als MimoChannelMatrix. GetPropagationChannelMatrix erhält die Dimensionen; GetPropagationCSI bleibt als flache, für 1x1 unveränderte Kompatibilitätsansicht bestehen.

5. SU-MIMO-MINIMALMODELL

5.1 SIGNALMODELL

Ohne Precoding gilt je Subcarrier

$$
\mathbf{y}[k] =
\sqrt{\frac{P}{N_s}}\,\mathbf{H}[k]\mathbf{x}[k] + \mathbf{n}[k],
$$

mit N_s = N_tx, E[x x^H] = I und räumlich weißem Rauschen E[n n^H] = N I. Die Eingangsgröße

$$
\rho = \frac{P}{N}
$$

ist die gesamte Referenz-SNR vor der Small-Scale-CFR-Gewichtung. Das Modell verlangt in dieser Unterphase N_rx >= N_tx. Overloaded MIMO wird erst mit einer expliziten Streamauswahl zugelassen.

5.2 LINEARER MMSE-EMPFÄNGER

Mit alpha = rho/N_s und G[k] = H[k]^H H[k] folgt die Fehlerkovarianz des linearen MMSE-Detektors:

$$
\mathbf{E}[k] =
(\mathbf{I} + \alpha\mathbf{G}[k])^{-1}.
$$

Die Post-Processing-SINR des Streams s ist

$$
\gamma_s[k] = \frac{1}{E_{s,s}[k]} - 1.
$$

Die Formel berücksichtigt gewünschte Leistung, Inter-Stream-Interferenz und Rauschverstärkung gemeinsam. Sie ist auch für rangdefiziente H numerisch definiert, weil I + alpha G positiv definit ist.

5.3 SISO-GRENZFALL

Für N_tx = N_rx = 1 ergibt sich

$$
  E = \frac{1}{1 + \rho |h|^2},
  \qquad
  \gamma = \rho |h|^2.
$$

Damit ist der neue Kern im skalaren Grenzfall exakt kompatibel mit dem
üblichen SISO-SNR-Modell.

5.4 FREQUENZKOMPRESSION

Vor einer MCS-spezifischen Kalibrierung wird je Stream eine
Mutual-Information-erhaltende AWGN-SNR ausgewiesen:

$$
  \gamma_{\mathrm{eff},s} =
  \exp\left(
    \frac{1}{N_\mathrm{sc}}\sum_k \ln(1+\gamma_s[k])
  \right)-1.
$$

Die mittlere ideale spektrale Effizienz über alle Streams ist

$$
  \eta =
  \frac{1}{N_\mathrm{sc}}
  \sum_k\sum_s \log_2(1+\gamma_s[k]).
$$

Diese MI-Größe bleibt eine Validierungsmetrik und ist keine unmittelbare
802.11-Datenrate oder PER. Die HE-PER-Abbildung verwendet stattdessen die
MCS-spezifische EESM-Kompression aus Abschnitt 5.6.

5.5 REFERENZ-SNR AUS DEM LINK BUDGET

Die skalare Wideband-Dämpfung L_wb bleibt von der normalisierten
Small-Scale-CFR getrennt. Aus der gesamten Sendeleistung $P_\mathrm{tx,dBm}$
folgen Empfangsleistung und lineare Empfangsleistung

$$
  P_\mathrm{rx,dBm} = P_\mathrm{tx,dBm} - L_\mathrm{wb},
  \qquad
  P_\mathrm{rx} =
  10^{(P_\mathrm{rx,dBm}-30)/10}\ \mathrm{W}.
$$

Mit Rauschbandbreite B in Hz, Temperatur T in Kelvin und
Empfängerrauschzahl NF in dB gelten

$$
  N_\mathrm{th} = kTB,
  \qquad
  F = 10^{NF/10},
  \qquad
  N_\mathrm{rx} = FkTB,
$$

und damit

$$

$$
  \rho =
  \frac{P_\mathrm{rx}}{N_\mathrm{rx}}
  =
  \frac{10^{(P_\mathrm{tx,dBm}-L_\mathrm{wb}-30)/10}}
    {kTB\,10^{NF/10}}.
$$

WifiLinkBudget gibt Empfangsleistung, thermisches Rauschen,
Empfängerrauschen und ρ separat aus. dB-/Linear- und dBm-/Watt-
Umrechnungen delegieren an DbToRatio, RatioToDb und DbmToW aus
ns-3.40 wifi-utils. Für $k=1{,}3803\cdot10^{-23}\ \mathrm{J/K}$ wird bewusst
derselbe Zahlenwert wie in ns-3.40 InterferenceHelper verwendet. Das ist die
Provenienz dieser physikalischen Konstante; es ist keine Fit-Konstante.
Anders als InterferenceHelper ist T hier ein expliziter positiver
Eingabeparameter und nicht fest auf 290 K gesetzt.

P_tx ist die Gesamtleistung über alle Spatial Streams; die
Aufteilung durch N_s erfolgt erst im MMSE-Modell. B ist die
Empfänger-Rauschbandbreite. Link Budget und MMSE addieren selbst keine externe
Interferenz; der SpectrumWifiPhy-Pfad bewertet sie anschließend skalar gemäß
Abschnitt 5.8. Ein zusätzlicher pauschaler RX-Diversity-Gewinn wird nicht
aufgeschlagen: Array- und Detektorgewinn sind bereits in $\mathbf H[k]$ und
den Post-MMSE-SINRs enthalten.

5.6 MCS-ABHÄNGIGE EESM-KOMPRESSION

Für HE MCS m komprimiert EESM alle $N_\mathrm{sc}N_s$
Post-MMSE-SINRs mit gleicher Gewichtung:

$$
  \gamma_{\mathrm{EESM},m}
  =
  -\beta_m \ln\left(
    \frac{1}{N_\mathrm{sc}N_s}
    \sum_{k=0}^{N_\mathrm{sc}-1}
    \sum_{s=0}^{N_s-1}
    \exp\left(-\frac{\gamma_s[k]}{\beta_m}\right)
  \right).
$$

$\beta_m>0$ hat wie γ lineare SINR-Einheiten und ist
MCS-abhängig. SetEesmBeta erwartet den linearen Wert; SetEesmBetaDb ist die
dB-Komfortfunktion. Zur Vermeidung numerischen Unterlaufs wird mit
$\gamma_\min=\min_{k,s}\gamma_s[k]$ äquivalent gerechnet:

$$
  \gamma_{\mathrm{EESM},m}
  =
  \gamma_\min
  -\beta_m \ln\left(
    \frac{1}{N_\mathrm{sc}N_s}
    \sum_{k,s}
    \exp\left(-\frac{\gamma_s[k]-\gamma_\min}{\beta_m}\right)
  \right).
$$

Für einen flachen Kanal mit identischen SINRs ist das Ergebnis unabhängig von
β_m exakt diese SINR. Das ermöglicht die SISO-AWGN-Paritätsprüfung.

Es werden bewusst keine Default-Betas ausgeliefert. Jeder HE MCS von 0 bis 11
muss vor der Nutzung explizit konfiguriert werden. Damit enthält der Code keine

unbelegten Kalibrierkonstanten. Die in analytischen Tests verwendeten positiven Werte sind ausschließlich Formel-Sentinels und ausdrücklich keine 802.11ax-Kalibrierung. Belastbare β_m müssen später aus dokumentierten Link-Level-Kurven gefittet und zusammen mit Datensatz, PHY-Konfiguration, Codeart und Paketlänge versioniert werden. Die bisherige MI-Kompression je Stream bleibt als modellunabhängige Diagnosemetrik erhalten und wird nicht als kalibrierte PER-Abbildung ausgegeben.

5.7 ABBILDUNG AUF NS-3-PER

HeSuMimoLinkAbstraction übernimmt WifiTxVector einschließlich HE-MCS, NSS und Codeart. NSS muss der Zahl der übertragenen Streams entsprechen. Nach EESM wird die vorhandene Schnittstelle unverändert verwendet:

$$
\begin{aligned}
&P_{\mathrm{succ}} = \\
&\mathrm{ErrorRateModel::GetChunkSuccessRate}\\
&(m,\mathrm{TxVector},\gamma_{\mathrm{EESM},m},8L,N_{\mathrm{rx}}),\\
&\qquad
PER = 1-P_{\mathrm{succ}},
\end{aligned}
$$

wobei L die Paketlänge in Byte ist. ErrorRateModel ist als ns-3-Attribut und über SetErrorRateModel konfigurierbar. Standard ist TableBasedErrorRateModel, also das ns-3.40-Defaultmodell für 802.11n/ac/ax.

Die Kalibrierprovenienz bleibt vollständig bei ns-3: Laut src/wifi/doc/source/wifi-design.rst stammen die AWGN-PER-Tabellen aus End-to-End-Linksimulationen mit MATLAB WLAN Toolbox, wurden gegen IEEE-TGn-Ergebnisse validiert und verwenden die Paketlängenextrapolation aus IEEE P802.11 TGax 11-14/0803r1. BCC besitzt Referenztabellen für 32 und 1458 Byte mit einer 400-Byte-Schwelle und MCS 0 bis 9; MCS 10/11 fällt auf YansErrorRateModel zurück. LDPC besitzt eine 1458-Byte-Referenz für MCS 0 bis 11. Die neue Abstraktion kopiert oder verändert keine dieser Kurven und fügt keinen SNR-Offset hinzu. Da sie Erfolgswahrscheinlichkeiten und keine Zufallsentscheidung ausgibt, ist die Berechnung bei identischen Eingaben deterministisch.

5.8 EINKOPPLUNG IN SPECTRUMWIFIPHY

Die Integration bleibt auf SionnaSpectrumPropagationLossModel::DoCalcRxPowerSpectralDensity beschränkt. Für einen HE-SU-PPDU liest sie N_s ausschließlich mit WifiTxVector::GetNss. Vor der Link-Abstraktion müssen $N_s=N_{\mathrm{tx}}$, die physikalischen TX-/RX-Antennenzahlen gleich den CFR-Dimensionen, die maximal unterstützten Spatial Streams mindestens N_s und $N_{\mathrm{ant,rx}}\geq N_s$ sein. Inkonsistenzen brechen damit vor MMSE, EESM und Paketempfang ab. $N_s<N_{\mathrm{tx}}$, Streamauswahl und Precoding gehören weiterhin ausdrücklich nicht zu Phase 1.

Der SpectrumChannel hat den skalaren Wideband-Pfadverlust bereits auf params->psd angewendet. Die MIMO-Integration verwendet daher

$$
\begin{aligned}
&P_{\mathrm{in}} = \\
&\int S_{\mathrm{params}}(f)\,\mathrm df \; G_{\mathrm{rx}}
\end{aligned}
$$

als bereits pfadverlustbehaftete Empfangsleistung. Der Wideband-Verlust aus dem Cache wird an dieser Stelle nur noch für den bestehenden Diagnose-Tag gelesen und nicht erneut multipliziert. Empfangsverstärkung,

Rauschbandbreite, 290 K und Empfängerrauschzahl gehen genau einmal in das Link Budget ein. Weil ns-3.40 die Rauschzahl nicht lesbar als WifiPhy-Attribut bereitstellt, verlangt SetReceiverNoiseFigureDb einen expliziten Wert; er muss mit SpectrumWifiPhy::RxNoiseFigure identisch konfiguriert werden. Ein fehlender Wert bricht den MIMO-Pfad früh ab.

MMSE und EESM liefern γ_eff. SpectrumWifiPhy erwartet dagegen weiterhin eine skalare PSD und wendet seine InterferenceHelper-Berechnung später selbst an. Für das AWGN-Fehlermodell ist dessen interner Diversity-Faktor

$$
  G_\mathrm{AWGN} =
  \begin{cases}
    N_\mathrm{ant,rx}/N_s, & N_\mathrm{ant,rx}>N_s,\\
    1, & \text{sonst}.
  \end{cases}
$$

Da der MMSE-Schritt den Arraygewinn bereits enthält, wird die äquivalente skalare Empfangsleistung auf

$$
  P_\mathrm{eq} =
  \frac{\gamma_\mathrm{eff} P_N}{G_\mathrm{AWGN}},
  \qquad P_N = kTB\,10^{NF/10}
$$

gesetzt. InterferenceHelper erhält damit bei zusätzlicher skalarer Interferenz I

$$
  \gamma_\mathrm{phy}
  = \frac{P_\mathrm{eq}}{P_N+I}G_\mathrm{AWGN}
  = \gamma_\mathrm{eff}\frac{P_N}{P_N+I}.
$$

Ohne Interferenz gilt exakt $\gamma_\mathrm{phy}=\gamma_\mathrm{eff}$; Pfadverlust, thermisches Rauschen, Rauschzahl und Arraygewinn werden nicht doppelt angewendet. Zeitvariable Interferenz wirkt weiterhin über SpectrumWifiPhy, jedoch als skalare Nachbewertung und nicht als subcarrierabhängige Interferenzkovarianz im MMSE-Detektor.

Der bisherige 1x1-Pfad bleibt unverändert skalar und benötigt kein EESM-Beta. Nicht-HE-Kontrollrahmen einer MIMO-PHY verwenden während Phase 1 das erste Antennenpaar, damit Beacon, Association und ACK ohne vorweggenommene Streamauswahl funktionieren. Der letzte Guard-Bin und das bestehende CFR-Tag-Format bleiben im 1x1-Fall kompatibel. Für automatisierte Beispiele kann eine normalisierte feste CFR den Sionna-Cache ersetzen; dieser deterministische Test-Hook ändert den Produktionspfad nicht. Der MimoRx-Trace meldet PPDU-UID, Knoten, Effective-SINR, PER und den kleinsten Kanalrang über alle Subcarrier.

Das Beispiel ns3_wifi_sumimo_minimal.cc erzeugt drei flache, normierte Kanäle bei HE MCS 5 und 20 dB Referenz-SNR: 1x1 mit $h=1$, gut konditioniertes 2x2 mit $\mathrm{diag}(\sqrt{2},\sqrt{2})$ und Rang-1-2x2 mit ausschließlich Einsen. Es misst realen UDP-Durchsatz über SpectrumWifiPhy und gibt zusätzlich PER, Stream-SINR, EESM-SINR und Mindestkanalrang aus. Der Aufruf muss ein positives --eesmBetaLinear liefern. Der im Regressionstest verwendete Wert 2 ist nur ein expliziter Formel-Sentinel; bei diesen flachen Kanälen ist das

Ergebnis betaunabhängig und stellt keine Produktionskalibrierung dar.

Die deterministische Referenzausgabe lautet:

  - 1x1: Rang 1, Stream-/Effective-SINR 20,000 dB, PER 0,000016 und
  68,1408 Mbit/s;
  - gut konditioniertes 2x2: Rang 2, beide Stream-SINRs und Effective-SINR
  20,000 dB, PER 0,000016 und 134,3904 Mbit/s;
  - Rang-1-2x2: Rang 1, beide Stream-SINRs und Effective-SINR -0,043214 dB,
  PER 1 und 0 Mbit/s.

6. VALIDIERUNG DER UNTERPHASEN 1.1 BIS 1.4

Die native Suite sionna enthält nun 15 schnelle Fälle:

1. Layouttest eines 2 x 2 x 2-Tensors mit eindeutig nummerierten Koeffizienten.
2. Reversible dB-/Linear-Umrechnung einschließlich 10 dB gleich Faktor zehn.
3. Link-Budget-Test für Sendeleistung, Pfadverlust, B, T und NF
   sowie die erwartete Halbierung der SNR bei Verdopplung von B oder T.
4. Flacher orthogonaler 2x2-Kanal: EESM erhält identische Tone-/Stream-SINRs
   unabhängig vom positiven Test-Beta.
5. Frequenzselektiver SISO-Kanal: EESM stimmt mit der analytischen
   Zweitonformel für das konfigurierte Beta überein.
6. SISO-Parität zur direkten ns-3.40-ErrorRateModel-Abfrage für HE MCS
   0, 3, 7 und 11, neun SNR-Punkte von -5 bis 35 dB sowie 32, 400 und
   1458 Byte. Das sind 108 Kombinationen; MCS 11 prüft auch den
   TableBased-Fallback auf Yans.
7. Monotone PER auf einem normierten frequenzselektiven Vier-Ton-Kanal für
   dieselben vier MCS und drei Paketgrößen bei 26 SNR-Punkten von -10 bis
   40 dB. Die 312 geprüften Werte bilden je Konfiguration eine nicht steigende
   Folge mit einem echten PER-Abfall.
8. Exakte Reproduzierbarkeit von Post-MMSE-SINR, EESM-SINR und PER über
   20 Wiederholungen eines komplexen frequenzselektiven 2x2-Kanals.
9. SISO-Test mit $h=2+j$ und $\rho=10$; erwartet wird $\gamma=50$.
10. Orthogonaler 2x2-Kanal $\mathbf H=\mathbf I$ und $\rho=20$; jeder
    Stream erhält $\gamma=10$ und $\eta=2\log_2(11)$.
11. Zwei diagonale Subcarrier diag(1,2) und diag(2,1) bei $\rho=8$;
    erwartet werden (4,16), (16,4) und die analytische MI-Kompression.
12. Deserialisierung einer alten Nachricht ohne Dimensionsfelder als
    $N_\mathrm{sc}\times1\times1$.
13. Protocol-Buffer-Rundlauf einer synthetischen 2 x 2 x 2-CFR mit den
    komplexen Werten 1-j bis 8-8j und indexgenauer Prüfung.
14. Mindestkanalrang eins beziehungsweise zwei für Rang-1- und orthogonale
    2x2-Kanäle über alle Subcarrier.
15. Spectrumadapter-Test für exaktes Rauschleistungs- und AWGN-Diversity-
    Compensation-Mapping ohne doppelte Anwendung von Rauschen oder Arraygewinn.

Die vier reinen Python-Vertragstests prüfen Standard- und explizite
Arraykonfiguration, bytegleiche 1x1-Serialisierung, die sc/rx/tx-Reihenfolge
einer synthetischen 2x2-CFR sowie die gemeinsame MIMO-Normalisierung. Diese
Tests benötigen weder Raytracing, ZMQ noch eine GPU. Die analytischen
Vergleiche des numerischen Kerns verwenden weiterhin eine absolute Toleranz
von 1e-10; die neuen Link-Budget-/EESM-Vergleiche verwenden 1e-12.

Der reale Server-Integrationstest konfiguriert 1x2-PlanarArrays an TX und RX,
führt Sionna RT auf der CPU aus und prüft die Form 2 x 2 x 256, 1024 komplexe
Werte und mittlere normierte Leistung eins. Die übrigen sieben Servertests
decken den unveränderten 1x1-Pfad, Mobilität, Kohärenzzeit und Mehrnutzer-
Anfragen ab.

Die ExampleAsTest-Suite führt den vollständigen UDP-/SpectrumWifiPhy-Pfad für

1x1, Vollrang-2x2 und Rang-1-2x2 aus und vergleicht alle ausgegebenen Metriken bytegenau mit der Referenzdatei. Ein separater Negativlauf bestätigt, dass das Beispiel ohne explizites EESM-Beta mit Status 2 abbricht.

6.1 AUSGEFÜHRTE CPU-VALIDIERUNG

Am 19. Juli 2026 wurden nach Abschluss der Unterphase 1.4 folgende Prüfungen erfolgreich ausgeführt:

- vollständiger Build mit Beispielen und Tests unter GCC 13.3
- native ns-3-Suite sionna: 15 von 15 Fällen in 0,009 s bestanden
- SISO-Paritätsmatrix: alle 108 Kombinationen stimmen innerhalb 1e-12 mit der direkt aufgerufenen ns-3-Fehlermodellschnittstelle überein
- frequenzselektive Monotonitätsmatrix: alle 312 PER-Werte bestanden
- Reproduzierbarkeit: 20 identische 2x2-Aufrufe mit exakt gleichen Ergebnissen
- ExampleAsTest sionna-example-sumimo-minimal: ein von einem Fall bestanden; Vollrang-2x2 erreicht 134,3904 statt 68,1408 Mbit/s bei 1x1, Rang-1-2x2 erreicht bei PER 1 erwartungsgemäß 0 Mbit/s
- Negativlauf ohne --eesmBetaLinear: dokumentierter Abbruch mit Status 2 statt eines impliziten Produktions-Betas
- reine Python-Vertragssuite: vier von vier Fällen in 0,001 s bestanden
- gesamte Sionna-CPU-Serversuite: acht von acht Fällen in 170,587 s bestanden; der reale 2x2-Fall lief in 1,574 s, lieferte 2 x 2 x 256 und erfüllte die Normalisierung innerhalb 1e-6
- P2P-End-to-End-Test example-sionna-sensing im unveränderten 1x1-Modus: drei CFR-Antworten, zwei empfangene UDP-Pakete zu je 1024 Byte und Cache-Hit-Rate 0,993056

Der ältere Beispielpfad example-sionna ist in der unveränderten Basis defekt: Er startet standardmäßig Mode 3 mit Submode 1, berechnet aber keinen Look-ahead-Zeitpunkt. Dadurch wird kein Receiver angelegt und Sionna meldet ValueError: Scene has no receivers. Der explizite P2P-Beispielpfad ist davon nicht betroffen. Eine Korrektur gehört in eine eigene kleine Baseline-Subphase.

7. BEKANNTE GRENZEN UND NÄCHSTE ENTSCHEIDUNGSPUNKTE

- Die Arraykonfiguration ist global für Sender und Empfänger. Knotenbezogene Geometrie und Orientierung gehören zur High-Level-API in Phase 2.
- Der integrierte MIMO-Pfad unterstützt nur HE SU mit einem Open-Loop-Stream je TX-Antenne. Nicht-HE-Kontrollrahmen verwenden das erste Antennenpaar; MU-MIMO, Streamauswahl und Precoding bleiben außerhalb von Phase 1.
- Zeitvariable externe Interferenz wird nach der MMSE-/EESM-Abstraktion durch InterferenceHelper skalar berücksichtigt. Eine frequenz- und antennenabhängige Interferenzkovarianz im MMSE-Detektor ist noch nicht modelliert.
- Die Rauschzahl muss am Adapter und an SpectrumWifiPhy explizit identisch gesetzt werden, da ns-3.40 das gesetzte WifiPhy-Attribut nicht lesbar macht.
- Es existiert absichtlich keine mitgelieferte HE-EESM-Beta-Tabelle. Frequenzselektive absolute PER-Prognosen benötigen vor der wissenschaftlichen Nutzung eine dokumentierte Link-Level-Kalibrierung. Ohne diese Konfiguration bricht das Modell früh ab.
- TableBasedErrorRateModel ist eine AWGN-Referenz. Seine SNR-/PER-Kurven und Paketlängenextrapolation stammen aus ns-3; ihre Grenzen gelten unverändert.
- Perfekte RX-CSI ist zunächst bewusst idealisiert.
- Pro Antenne wird genau ein Stream angenommen; Streamauswahl und \(N_s<N_\mathrm{tx}\) gehören zur Precoding-Phase.
- Die MI-Kompression bleibt eine Diagnosemetrik; nur die explizit konfigurierte EESM-Ausgabe wird an ErrorRateModel übergeben.
- Die bestehende Cache-Schlüsselbildung nimmt Reziprozität an. Für nicht-reziprokes Precoding müssen Richtung und Konfiguration Teil des Schlüssels werden.

Unterphase 1.4 ist abgeschlossen; damit ist das abgegrenzte SU-MIMO-
Minimalmodell der Phase 1 vollständig. Phase 2, Streamauswahl und Precoding
wurden bewusst nicht begonnen.

8. AUSFÜHRUNG

Vom ns-3.40-Verzeichnis:

```
./ns3 configure --enable-examples --enable-tests
./ns3 build
./test.py -s sionna
./test.py -s sionna-example-sumimo-minimal
./ns3 run "ns3_wifi_sumimo_minimal --eesmBetaLinear=2"
```

Python-Vertrag und Server vom Verzeichnis contrib/sionna/model/ns3sionna:

```
conda run -n sionna-venv python -m unittest -v tests.test_mimo_data_contract
conda run -n sionna-venv python -m unittest -v tests.unit_tests
```

PDF neu erzeugen:

```
conda run -n sionna-venv python
contrib/sionna/tools/render_model_pdf.py
contrib/sionna/ns3sionna_mimo_model.md
contrib/sionna/ns3sionna_mimo_model.pdf
```

Das Rendering-Skript benötigt nur Matplotlib und läuft im vorhandenen
CPU-Environment.